

Investigation 1 – Gradients and Matrix Completion

Jonathon Hare & Antonia Marcu
 {j.s.hare, a.marcu}@soton.ac.uk

2026–27

This is the first of four **individual** experimental investigations that make up the laboratory portfolio. It should be completed after the automatic differentiation and optimisation practicals.

You should write up your results, answers, and findings succinctly, using *no more than two sides of A4* for this investigation. Code need not be reproduced in full: include only short snippets needed to explain your method. This exercise is worth 10 marks (10% of the module).

1 Low-rank factorisation with automatic differentiation

Consider the optimisation problem

$$\min_{\hat{\mathbf{U}}, \hat{\mathbf{V}}} \frac{1}{2} \left\| \mathbf{A} - \hat{\mathbf{U}} \hat{\mathbf{V}}^\top \right\|_{\text{F}}^2, \quad (1)$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\hat{\mathbf{U}} \in \mathbb{R}^{m \times r}$, $\hat{\mathbf{V}} \in \mathbb{R}^{n \times r}$, and $r < \min(m, n)$. Use

$$\mathbf{A} = \begin{bmatrix} 0.3374 & 0.6005 & 0.1735 \\ 3.3359 & 0.0492 & 1.8374 \\ 2.9407 & 0.5301 & 2.2620 \end{bmatrix}.$$

Set the PyTorch random seed to zero in all experiments unless instructed otherwise.

1.1 Implement an autograd factorisation (2 marks)

Implement a function that initialises $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$ with uniform random values and minimises the objective using PyTorch automatic differentiation and gradient descent. Implement the gradient descent yourself, rather than using a PyTorch optimizer. Your function should return both factors and a record of the loss. Use rank 2 and state the learning rate and number of iterations used.

Note that this is *not* intended to be the stochastic/streaming implementation discussed in the lecture, which visits one observed matrix element at a time. At each iteration you should compute the objective using the whole matrix $\mathbf{A}$, then jointly optimise all the entries of $\hat{\mathbf{U}}$ and $\hat{\mathbf{V}}$.

Report the final reconstruction and loss. Include the part of your training loop that clears gradients, computes the loss, runs reverse-mode differentiation, and updates the factors.

2 Derivative products and gradient checking

Define

$$f(\mathbf{U}, \mathbf{V}) = \mathbf{U} \mathbf{V}^\top$$

and let

$$\mathcal{L}(\mathbf{U}, \mathbf{V}) = \frac{1}{2} \left\| f(\mathbf{U}, \mathbf{V}) - \mathbf{A} \right\|_{\text{F}}^2.$$

For this section use double precision and set the PyTorch random seed to zero. Generate the “direction” tensors with `dU = torch.randn_like(U)` and `dV = torch.randn_like(V)`. Thus each entry is drawn independently from the standard normal distribution and each tensor matches its input’s shape, data type, and device. Treat $(\hat{\mathbf{U}}, \hat{\mathbf{V}})$ as one direction in the joint input space; you do not need to normalise it.

2.1 Check JVPs, VJP, and backward (3 marks)

Use `torch.func.jvp` to compute the change in f in the direction $(\dot{\mathbf{U}}, \dot{\mathbf{V}})$. Compare it with a symmetric finite-difference approximation and report the maximum absolute difference for at least three suitably chosen step sizes.

Next use `torch.func.vjp` with the cotangent $f(\mathbf{U}, \mathbf{V}) - \mathbf{A}$. Compare the resulting tensors with the gradients produced by calling `backward()` on $\mathcal{L}$. Report an appropriate numerical measure of agreement.

Briefly explain what the tangent, cotangent, JVP, and two returned VJP tensors represent in this example, including their shapes.

3 Comparison with SVD

3.1 Compare with a truncated SVD (2 marks)

Use `torch.linalg.svd` to construct the best rank-2 reconstruction of $\mathbf{A}$. Report its reconstruction loss and compare it with the result from Task 1.1. Explain the difference, including why optimising the factorisation does not necessarily reproduce the same factors even when it reaches the same reconstruction.

4 Matrix completion

Let

$$\mathbf{M} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

be a binary observation mask. Entries for which $M_{ij} = 0$ are to be treated as unobserved and must not contribute to the loss.

4.1 Complete a partially observed matrix (2 marks)

Adapt your method from Task 1.1 to optimise only the observed entries of $\mathbf{A}$. Report the reconstructed matrix, the loss on observed entries, and the estimates for the two held-out entries. Comment briefly on what this small example does and does not demonstrate about matrix completion.

4.2 Predict a movie rating (1 mark)

The supplied `ratings.pt` (<https://tinyurl.com/ratings-pt>) tensor contains ratings of 1,000 films by 10,000 Netflix users. Rows correspond to films, columns correspond to users, and a zero represents a missing rating. The supplied `titles.csv` (<https://tinyurl.com/titles-csv>) file maps row indices to film titles.

For this task, factorise the raw ratings as supplied: do not mean-centre the ratings, add user or film bias terms, or add regularisation. These are useful extensions in a practical recommender system, but each would change the optimisation problem and the expected numerical result here.

Use your jointly optimised masked factorisation with rank 5, 1,000 iterations, and a learning rate of 10^{-5} . What rating do you predict that the fifth user (column index 4) would give to “A Beautiful Mind”? Also report the sum of squared errors over the observed entries of the ratings matrix.